SQL WINDOWS FUNCTIONS

A window function performs a calculation on a group of rows that are related to the current row, without combining those rows into one row.

General Syntax:

```
FUNCTION () OVER
(
PARTITION BY column
ORDER BY column
)
```

There are three important parts in the above syntax:

a. FUNCTION
   - ROW_NUMBER ()
   - RANK ()
   - DENSE_RANK ()
   - LAG ()
   - LEAD ()
b. PARTITION BY
   - Divides dataset into groups.
c. ORDER BY
   - Decides the order in which the given window function should operate.

FUNCTIONS:

1) ROW_NUMBER ()

- ✓ It gives a **unique sequential number** to every row.
- ✓ Example:

```
SELECT
   Name,
   Marks,
   ROW_NUMBER () OVER (ORDER BY Marks DESC) AS Row_No
FROM STUDENT;
```

- ✓ Here in the above example, highest marks gets Row Number as 1.
- ✓ **ROW_NUMBER () never gives the same number to two rows.**

2) RANK ()

- ✓ It is used to assign rankings.
- ✓ Example:

```
SELECT
   Name,
   Marks,
   RANK () OVER (ORDER BY Marks DESC) AS Rank
FROM STUDENT;
```

Name	Marks	Rank
A	98	1
B	96	2
C	96	2
D	92	4
E	90	5

- ✓ **RANK () leaves gaps after duplicate values.**

3) DENSE_RANK ()

- ✓ It is also used for ranking.
- ✓ Example:

```
SELECT
   Name,
   Marks,
   DENSE_RANK () OVER (ORDER BY Marks DESC) AS Dense_Rank
FROM STUDENT;
```

Name	Marks	Rank
A	98	1
B	96	2
C	96	2
D	92	3
E	90	4

- ✓ **DENSE_RANK () never leaves gaps after duplicate values.**

4) LAG ()

- ✓ It is used to get the value from the **previous row**.
- ✓ Example:

```
SELECT
   Name,
   Price,
   LAG (Price) OVER (ORDER BY Name) AS PreviousPrice
FROM PRODUCT;
```

Name	Price	PreviousPrice
A	150	NULL
B	200	150
C	300	200
D	450	300
E	110	450

5) LEAD ()

- ✓ It is used to get the value from the **next row**.
- ✓ Example:

```
SELECT
   Name,
   Price,
   LEAD (Price) OVER (ORDER BY Name) AS NextPrice
FROM PRODUCT;
```

Name	Price	NextPrice
A	150	200
B	200	300
C	300	450
D	450	110
E	110	NULL

PARTITION BY

- ✓ It divides the table into separate groups.

Example:

Student	House	Marks
A	Red	80
B	Red	70
C	Blue	90
D	Blue	60

```
SELECT
    Student,
    House,
    Marks,
    RANK () OVER (
        PARTITION BY House
        ORDER BY Marks DESC
    ) AS HouseRank
FROM Student;
```

OUTPUT:

Student	House	Marks	HouseRank
A	Red	80	1
B	Red	70	2
C	Blue	90	1
D	Blue	60	2

ORDER BY

Example:

a. RANK () OVER (ORDER BY Marks DESC)
   - Rank students from highest marks to lowest marks.

b. RANK () OVER (ORDER BY Marks ASC)
   - Rank students from lowest marks to highest marks.